

**LAPORAN PRAKTIKUM ALGORITMA DAN
PEMROGRAMAN 2**

WEEK 14

“Sorting ”



Disusun Oleh :

Farel Tri Julian
109082500163

Dosen :

WAHYU ANDI SAPUTRA, S.Pd., M.Eng

**PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026**

Dasar teori

Pencarian (searching) adalah proses menemukan lokasi atau keberadaan suatu data (dapat berupa key atau value) di dalam sekumpulan data yang tersimpan dalam memori, seperti dalam struktur data array. Di dalam mata kuliah Algoritma dan Pemrograman, proses pencarian data umumnya dibagi menjadi dua metode utama berdasarkan kondisi data awal, yaitu:

1. Pencarian Berurutan (Sequential / Linear Search)

Metode ini digunakan apabila sekumpulan data yang dimiliki berada dalam kondisi acak atau belum terurut. Prosesnya dilakukan dengan cara memeriksa setiap elemen array satu per satu, mulai dari indeks pertama hingga indeks terakhir, atau sampai data yang dicari berhasil ditemukan. Kelebihan metode ini adalah implementasinya yang sangat sederhana, namun kurang efisien untuk jumlah data dalam skala besar karena memiliki kompleksitas waktu terbaik dan terburuk.

2. Pencarian Biner (Binary Search / Belah Dua)

Pencarian biner merupakan metode pencarian yang jauh lebih efisien, namun memiliki syarat mutlak: data di dalam array harus sudah dalam keadaan terurut, baik secara menaik (ascending) maupun menurun (descending).

Prinsip kerja dari algoritma Binary Search adalah dengan membagi dua ruang pencarian secara berulang (belah dua):

- Pertama, program menentukan posisi indeks tengah (mid) dari array.
- Nilai pada posisi tengah tersebut dibandingkan dengan data target yang dicari.
- Jika nilai tengah sama dengan target, maka pencarian selesai.
- Jika data target lebih kecil atau lebih besar (tergantung pada urutan ascending atau descending), setengah dari sisa data akan dieliminasi, dan pencarian dilanjutkan pada setengah bagian data lainnya dengan memperbarui batas atas atau batas bawah indeks secara dinamis.

Karena ruang pencarian selalu dipotong setengah pada setiap tahapannya, Binary Search memiliki kompleksitas waktu yang sangat efisien, yaitu $\log n$. Oleh karena itu, dalam penerapannya pada program yang kompleks (seperti pengolahan data perpustakaan), sebuah array biasanya akan diurutkan terlebih dahulu menggunakan metode sorting (seperti Insertion Sort atau Selection Sort) sebelum dilakukan operasi Binary Search agar proses pencarian berjalan optimal.

WEEK 14

Tugas 1

```
package main

import "fmt"

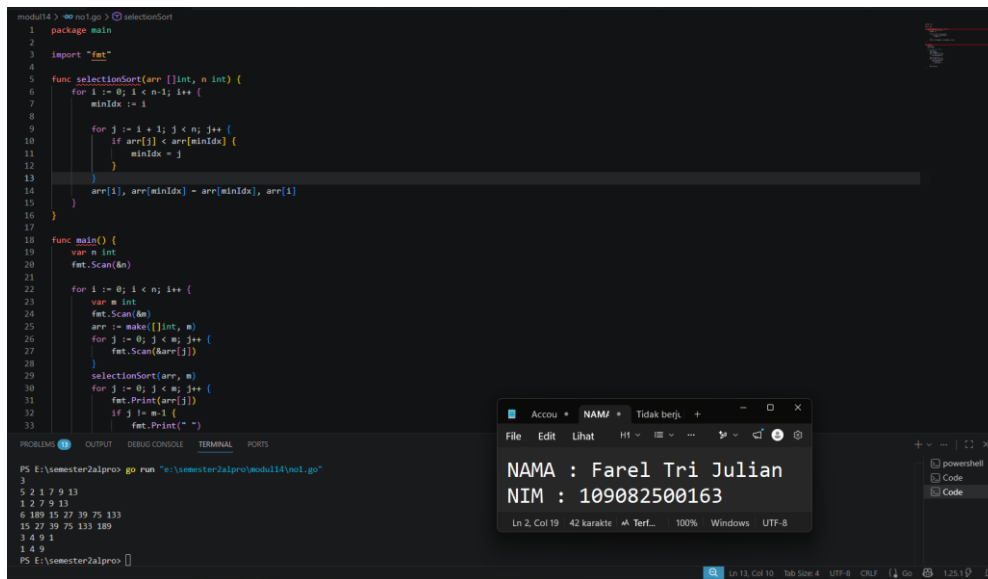
func selectionSort(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        minIdx := i

        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }
        arr[i], arr[minIdx] = arr[minIdx], arr[i]
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)
        arr := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&arr[j])
        }
        selectionSort(arr, m)
        for j := 0; j < m; j++ {
            fmt.Print(arr[j])
            if j != m-1 {
                fmt.Print(" ")
            }
        }
        fmt.Println()
    }
}
```

Screenshots Output



The screenshot shows a Go IDE with a file named `modul4.go` containing the following code:

```
1 package main
2
3 import "fmt"
4
5 func selectionSort(arr []int, n int) {
6     for i := 0; i < n-1; i++ {
7         minIdx := i
8
9         for j := i + 1; j < n; j++ {
10             if arr[j] < arr[minIdx] {
11                 minIdx = j
12             }
13         }
14         arr[i], arr[minIdx] = arr[minIdx], arr[i]
15     }
16 }
17
18 func main() {
19     var n int
20     fmt.Scan(&n)
21
22     for i := 0; i < n; i++ {
23         var m int
24         fmt.Scan(&m)
25         arr := make([]int, m)
26         for j := 0; j < m; j++ {
27             fmt.Scan(&arr[j])
28         }
29         selectionSort(arr, m)
30         for j := 0; j < m; j++ {
31             fmt.Print(arr[j])
32             if j != m-1 {
33                 fmt.Print(" ")
34             }
35         }
36     }
37 }
```

The terminal output shows the program running and displaying the sorted array:

```
PS E:\semester2alpro> go run "e:\semester2alpro\modul4\mod4.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
1 4 9 1
1 4 9
```

A small window titled "NAMA : Farel Tri Julian" and "NIM : 109082500163" is also visible in the foreground.

Deskripsi :

Program tersebut digunakan untuk mengurutkan data nomor rumah pada setiap daerah menggunakan algoritma Selection Sort. Program membaca jumlah daerah, kemudian untuk setiap daerah membaca banyaknya data dan nilai nomor rumah yang dimasukkan ke dalam array. Setelah itu, fungsi `selectionSort` akan mencari nilai terkecil pada array dan menukarnya secara berulang hingga seluruh data terurut dari kecil ke besar.

Tugas 2

```
package main

import "fmt"

func selectionSort(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        minIdx := i

        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }

        arr[i], arr[minIdx] = arr[minIdx], arr[i]
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
```

```

var m int
fmt.Scan(&m)

arr := make([]int, m)

for j := 0; j < m; j++ {
    fmt.Scan(&arr[j])
}

selectionSort(arr, m)

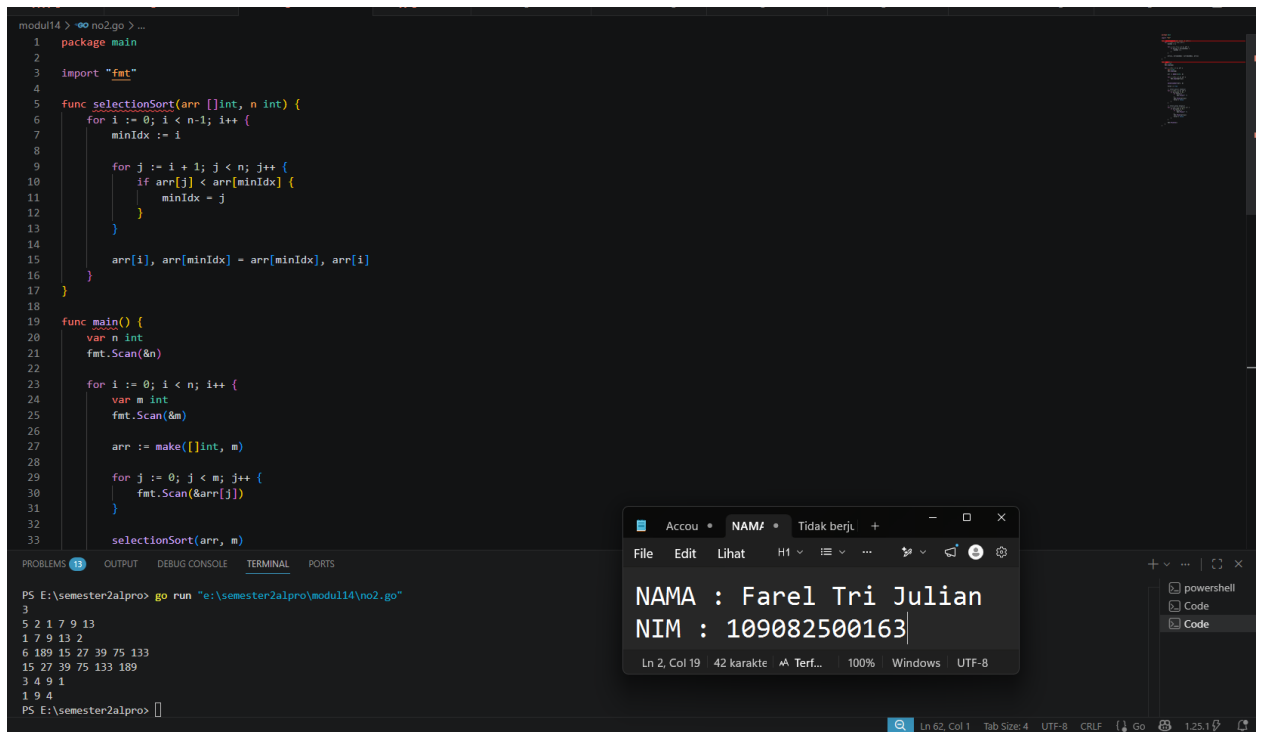
first := true

// Cetak ganjil membesar
for j := 0; j < m; j++ {
    if arr[j]%2 != 0 {
        if !first {
            fmt.Print(" ")
        }
        fmt.Print(arr[j])
        first = false
    }
}

for j := m - 1; j >= 0; j-- {
    if arr[j]%2 == 0 {
        if !first {
            fmt.Print(" ")
        }
        fmt.Print(arr[j])
        first = false
    }
}

fmt.Println()
}

```



```
modul14 > go no2.go > ...
1 package main
2
3 import "fmt"
4
5 func selectionSort(arr []int, n int) {
6     for i := 0; i < n-1; i++ {
7         minIdx := i
8
9         for j := i + 1; j < n; j++ {
10             if arr[j] < arr[minIdx] {
11                 minIdx = j
12             }
13         }
14
15         arr[i], arr[minIdx] = arr[minIdx], arr[i]
16     }
17 }
18
19 func main() {
20     var n int
21     fmt.Scan(&n)
22
23     for i := 0; i < n; i++ {
24         var m int
25         fmt.Scan(&m)
26
27         arr := make([]int, m)
28
29         for j := 0; j < m; j++ {
30             fmt.Scan(&arr[j])
31         }
32
33         selectionSort(arr, m)
34     }
35 }
```

PS E:\semester2alpro> go run "e:\semester2alpro\modul14\no2.go"

```
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS E:\semester2alpro>
```

NAMA : Farel Tri Julian
NIM : 109082500163

Deskripsi :

Program tersebut digunakan untuk mengurutkan nomor rumah menggunakan algoritma Selection Sort, kemudian menampilkan bilangan ganjil terlebih dahulu secara menaik dan dilanjutkan dengan bilangan genap secara menurun. Program membaca jumlah daerah, banyak data pada setiap daerah, serta nomor rumah yang disimpan dalam array. Setelah data diurutkan dari kecil ke besar, program mencetak semua bilangan ganjil terlebih dahulu, kemudian mencetak bilangan genap dari nilai terbesar ke terkecil sesuai ketentuan soal.

Tugas 3

```
package main

import "fmt"

const NMAX int = 100000

type arrInt [NMAX]int

func insertionSort(A *arrInt, n int) {
    for i := 1; i < n; i++ {
        temp := A[i]
        j := i - 1

        for j >= 0 && A[j] > temp {
            A[j+1] = A[j]
            j--
        }

        A[j+1] = temp
    }
}

func median(A arrInt, n int) int {
    if n%2 == 1 {
        return A[n/2]
    }
    return (A[(n/2)-1] + A[n/2]) / 2
}

func main() {
    var A arrInt
    var x, n int

    n = 0

    for {
        fmt.Scan(&x)

        if x == -5313 {
            break
        }

        if x == 0 {
            insertionSort(&A, n)
            fmt.Println(median(A, n))
        } else {
            A[n] = x
            n++
        }
    }
}
```

```
}  
}
```

Screenshots Output

```
modul14 > go no3.go > main  
1 package main  
2  
3 import "fmt"  
4  
5 const NMAX int = 100000  
6  
7 type arrInt [NMAX]int  
8  
9 func insertionSort(A *arrInt, n int) {  
10     for i := 1; i < n; i++ {  
11         temp := A[i]  
12         j := i - 1  
13  
14         for j >= 0 && A[j] > temp {  
15             A[j+1] = A[j]  
16             j--  
17         }  
18         A[j+1] = temp  
19     }  
20 }  
21  
22  
23 func median(A arrInt, n int) int {  
24     if n%2 == 1 {  
25         return A[n/2]  
26     }  
27     return (A[(n/2)-1] + A[n/2]) / 2  
28 }  
29  
30 func main() {  
31     var A arrInt  
32     var x, n int  
33  
34     for i := 0; i < NMAX; i++ {  
35         if i%10 == 0 {  
36             fmt.Println("Input 10 numbers")  
37         }  
38         x, _ = fmt.ScanInt()  
39         A[i] = x  
40         if x == -5313 {  
41             break  
42         }  
43     }  
44     n = i  
45     insertionSort(&A, n)  
46     fmt.Println("Median:", median(A, n))  
47 }
```

```
PS E:\semester2alpro> go run "e:\semester2alpro\modul14\n03.go"  
7 23 11 0 5 19 2 29 3 13 17 0 -5313  
11  
12  
PS E:\semester2alpro>
```

Deskripsi :

Program tersebut digunakan untuk membaca sekumpulan bilangan bulat dan menampilkan nilai median setiap kali pengguna memasukkan angka 0. Data yang dimasukkan akan disimpan ke dalam array, kemudian diurutkan menggunakan algoritma Insertion Sort sebelum median dihitung. Jika jumlah data ganjil, median diambil dari nilai tengah array, sedangkan jika jumlah data genap, median diperoleh dari rata-rata dua nilai tengah. Program akan terus berjalan hingga pengguna memasukkan angka -5313 sebagai penanda akhir input.

Tugas 4

```
package main

import "fmt"

const NMAX int = 100

type arrInt [NMAX]int

func insertionSort(A *arrInt, n int) {
    var temp, j int

    for i := 1; i < n; i++ {
        temp = A[i]
        j = i - 1

        for j >= 0 && A[j] > temp {
            A[j+1] = A[j]
            j--
        }

        A[j+1] = temp
    }
}

func jarakTetap(A arrInt, n int) bool {
    if n <= 2 {
        return true
    }

    selisih := A[1] - A[0]

    for i := 2; i < n; i++ {
        if A[i]-A[i-1] != selisih {
            return false
        }
    }

    return true
}

func main() {
    var A arrInt
    var x, n int

    n = 0
    for {
        fmt.Scan(&x)

        if x < 0 {
```

```

        break
    }

    A[n] = x
    n++
}

insertionSort(&A, n)

for i := 0; i < n; i++ {
    fmt.Print(A[i])

    if i != n-1 {
        fmt.Print(" ")
    }
}
fmt.Println()

if jarakTetap(A, n) {
    fmt.Println("Data berjarak tetap")
} else {
    fmt.Println("Data berjarak tidak tetap")
}
}

```

Screenshots Output

The screenshot shows a Go IDE with the following source code for `jarakTetap`:

```

modul14 > go mod init modul14
package main

import "fmt"

const NMAX int = 100

type arrInt [NMAX]int

func insertionSort(A *arrInt, n int) {
    var temp, j int
    for i := 1; i < n; i++ {
        temp = A[i]
        j = i - 1
        for j >= 0 && A[j] > temp {
            A[j+1] = A[j]
            j--
        }
        A[j+1] = temp
    }
}

func jarakTetap(A arrInt, n int) bool {
    if n <= 2 {
        return true
    }

    selisih := A[1] - A[0]

    for i := 2; i < n; i++ {
        if A[i]-A[i-1] != selisih {

```

The terminal output shows the execution of the program:

```

PS E:\semester2alpro> go run "e:\semester2alpro\modul14\n04.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak tetap
PS E:\semester2alpro> go run "e:\semester2alpro\modul14\n04.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS E:\semester2alpro>

```

An overlay window titled "NAMA" displays the user's name and NIM:

```

NAMA : Farel Tri Julian
NIM : 109082500163

```

Deskripsi :

Program tersebut digunakan untuk membaca sekumpulan bilangan bulat positif, mengurutkannya menggunakan algoritma Insertion Sort, lalu memeriksa apakah data memiliki jarak atau selisih yang tetap antar elemen setelah diurutkan. Input akan terus dibaca hingga pengguna memasukkan bilangan negatif sebagai tanda berhenti. Setelah proses pengurutan selesai, program menampilkan data yang telah terurut dan memberikan informasi apakah data tersebut memiliki selisih yang sama antar elemen (“Data berjarak tetap”) atau tidak (“Data berjarak tidak tetap”).

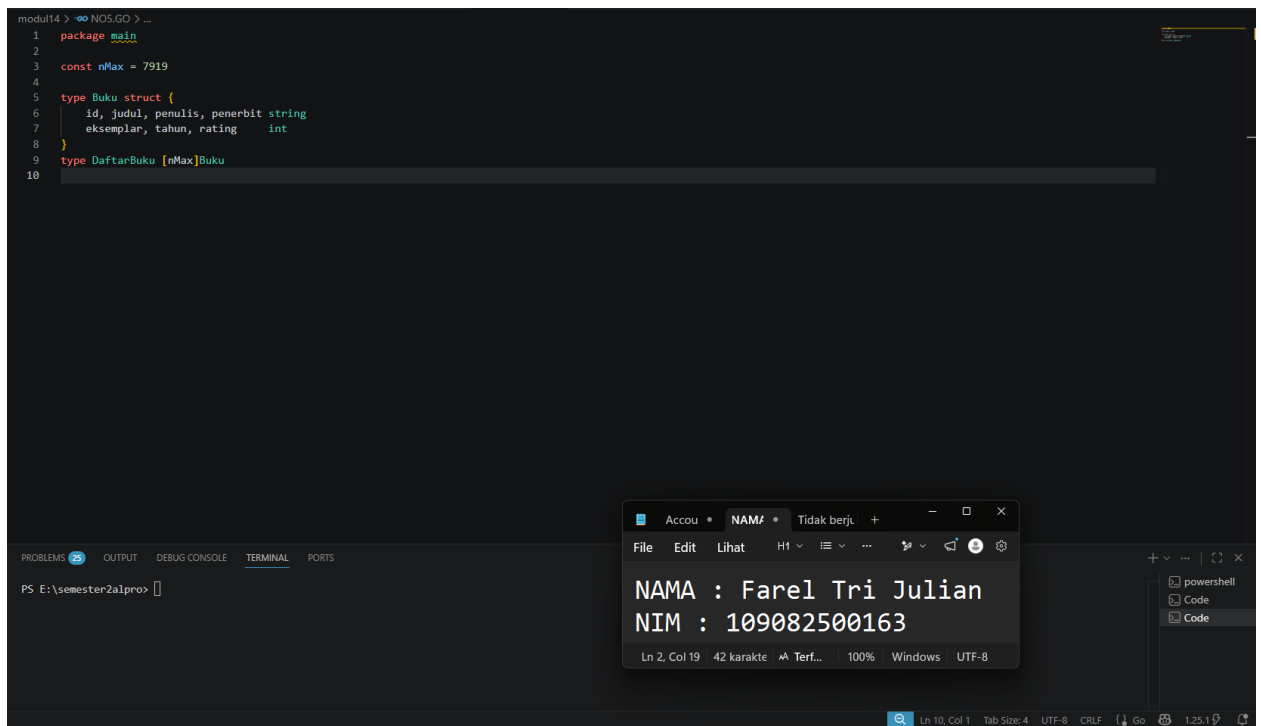
Tugas 5

```
package main

const nMax = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating    int
}
type DaftarBuku [nMax]Buku
```

Screenshots Output



The screenshot shows a Go IDE with a code editor and a terminal. The code editor displays the following Go code:

```
modul14 > NO5.GO > ...
1 package main
2
3 const nMax = 7919
4
5 type Buku struct {
6     id, judul, penulis, penerbit string
7     eksemplar, tahun, rating    int
8 }
9 type DaftarBuku [nMax]Buku
10
```

The terminal window shows the output of the program:

```
PS E:\semester2alpro>
NAMA : Farel Tri Julian
NIM : 109082500163
```

The terminal window also shows the status bar: Ln 2, Col 19, 42 karakter, 100% Windows, UTF-8.

Deskripsi :

Kode tersebut digunakan untuk mendefinisikan struktur data buku. Tipe data Buku menyimpan informasi seperti ID buku, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating buku. Selain itu, dibuat juga tipe data DaftarBuku, yaitu array yang mampu menyimpan hingga 7919 data buku menggunakan tipe Buku, sehingga memudahkan pengelolaan kumpulan data buku dalam program.

Daftar pustaka

Kadir, A., *Pengenalan Algoritma: Pendekatan Secara Visual dan Interaktif menggunakan Go*, Yogyakarta: Andi Offset, 2021.

Munir, R., *Algoritma dan Pemrograman dalam Bahasa Pascal, C, dan Go*, Bandung: Informatika Bandung, 2020.

Tim Dosen Alpro, *Modul Praktikum Algoritma dan Pemrograman 2: Modul 14 Sorting & Searching*, Purwokerto: Fakultas Informatika, Telkom University Purwokerto, 2026.

Donovan, A. A., & Kernighan, B. W., *The Go Programming Language*, New Jersey: Addison-Wesley Profile, 2015.